OBJECT – ORIENTED PROGRAMMING

Object - Oriented Programming is a programming paradigm in which computational problems are solved using Objects that resemble real-world entities. The Systems that follow such an approach are called Object – Oriented Programming Systems or OOPS.

Class

A Class is a description or blueprint of properties(data) and behavior(methods) of an object. The **`class`** keyword is used to create a class in Java. The name of a class should start with an uppercase letter.

The variables inside a class are called **member variables** whereas the functions inside a class are called **methods**. Together, they are referred to as the **members of a class**. A class is a logical construct and does not occupy memory.

The contents of a class are enclosed inside curly braces ({}). Conventionally, a class contains member variable declarations followed by method definitions.

```
class ClassName {
      //member variable declarations
      //method definitions
}
```

Object

An object is an instance of a class. It is the realization of a class and takes up space in the memory. An object is of the type of its class and is used to access the members of the class. The **dot(.)** operator is used as the member access operator.

The `new` keyword is used to create a new object. It creates an object dynamically(during run time) and returns a reference to it. This process is called **Dynamic memory allocation**. Objects are created inside the Heap memory.

```
//Object Creation
ClassName reference = new ClassName();

//member access
reference.memberVariableName;
reference.methodName();
```

Scope of variables

Local variables

These are the variables inside of a method. Their scope is limited to the method in which they are declared and they cannot be accessed outside the method.

Instance variables

These are the variables that are present inside of a class but outside of its method. These are also called **fields**. These variables are accessed using objects of the class. The value of these variables differs for each object of the class.

Block-level variables

These are the variables inside of a block, such as a loop or a decision making structure(`if, for, while`). Their scope exists only within the block in which they are declared.

```
class Student {
      // Instance variable - name, marks
      String name;
      int marks;

      void checkResult() {
            //Local variable - passMark
            int passMark = 40;

            //Block-level variable - result
            if(marks >= passMark) {
                  String result = "Pass";
                  System.out.print(result);
            }
            else {
                  String result = "Fail";
                  System.out.print(result);
            }
      }
}

class Main {
      public static void main(String[] args) {
            //First object
            Student s1 = new Student();
            s1.name = "Arjun";
            s1.marks = 75;

            //Second object
            Student s2 = new Student();
            s2.name = "Ravi";
            s2.marks = 30;

            s1.checkResult();  //Outputs Pass
            s2.checkResult();  //Outputs Fail
}
```

`'this'` keyword

`'this'` keyword refers to the current object of a class. It is primarily used to differentiate between the instance variables and local variables, especially inside of a constructor.

Constructor

A Constructor is a special method which is automatically called whenever an object of a class is created. It has no return type and is primarily used to initialize the instance variables of a class.

Default Constructor

A constructor with no parameters is called a **Default constructor** or non-parameterised constructor. When no constructor is defined, a default constructor is automatically generated by Java.

Parameterized Constructor

A **Parameterized constructor** is a constructor that accepts parameters and is used to initialize instance variables with specific values at the time of object creation.

Copy Constructor

A **Copy constructor** is a constructor that creates a new object by copying the values of another object of the same class.

`this()` method

The `this()` method is used inside a constructor to call another constructor of the same class.

```
class Student {
      int id;
      String name;
      //Default Constructor
      Student() {
            this(0, "NA");
      }
      //Parameterized Constructor
      Student(int id, String name) {
            this.id = id;
            this.name = name;
      }
      //Copy Constructor
      Student(Student original) {
            this(original.id, original.name);
      }
}
class Main {
      public static void main(String[] args) {
            //Calls Parameterized Constructor
            Student s1 = new Student(101, "Arav");
            //Calls Default Constructor
            Student s2 = new Student();
            //Calls Copy Constructor
            Student s3 = new Student(s1);
            System.out.print(s1.id + " " + s1.name); //O/P 101 Arav
            System.out.print(s2.id + " " + s2.name); //O/P 0 NA
            System.out.print(s3.id + " " + s3.name); //O/P 101 Arav
      }
}
```

The 'static' keyword

When a member is declared as static, only a single copy of the member is created and shared by all objects of that class. A change made to a static member is reflected across all instances of the class.

Static members can be accessed using Class name without creating objects. Non-static members can access static members directly. However, a static member cannot access non-static members without creating an object.

```
class Example {
      void displayNonStatic() {
            System.out.println("This is a non-static method");
      }
      static void displayStatic() {
            System.out.println("This is a static method");
      }
}

class Main {
      public static void main(String[] args) {
            Example eg = new Example();
            eg.displayNonStatic();      //needs an object
            Example.displayStatic();    //no need of an object
      }
}
```

Why `main()` method is static?

The `main()` method is the starting point of program execution. It is declared `static` so that JVM can call it without creating an object. Otherwise, the JVM would need to create an object before calling `main()`, which is not possible since program execution starts from `main()` itself.

`print()` method

`print()` is a non-static method of the `PrintStream` class. `out` is a static field of the `System` class and holds an object of type `PrintStream`. Hence the syntax,

```
System.out.print()
```

Java Garbage Collection

Garbage Collection in Java is an automatic process in which JVM frees the Heap memory by destroying unreachable(non-referenced) objects. This process only happens when the Heap memory needs to be freed.

`finalize()` method

Objects cannot be destroyed manually, However we can define certain actions to perform before an object is destroyed by using the `finalize()` method.

```java
class Example {
    static int count = 0;
    protected void finalize() {
        count++;
        System.out.println("finalize() called " + count);
    }
    public static void main(String[] args) {
        for (int i = 0; i < 10_000_000; i++) {
            // Many objects are created, but no references are stored.
            // These objects become unreachable immediately
            //and are eligible for garbage collection.
            new Example();
        }

        // main() and finalize() may execute on different threads.
        // The finalize() method can still be executed after main()
        //completes execution, as it is invoked by the Garbage Collector.
        System.out.println("End of main");
    }
}
```